Physical Design Laboratory Manual:
Gate-Level Simulation

Department of Electronics and Communication Engineering

1 Introduction

Once a digital block has successfully completed the RTL-to-GDSII flow, a critical step remains for a complete ASIC design cycle: the physical layout must be verified for timing correctness using Gate-Level Simulation (GLS) with back-annotated delays. This manual provides step-by-step instructions for this phase using the SkyWater 130nm PDK.

2 Gate-Level Simulation (GLS)

In RTL simulation, logic gates have zero delay. In Gate-Level Simulation, we simulate the exact standard cells placed on your chip and inject the physical wire delays extracted during routing.

2.1 Step 1.1: Locate the Extracted Files

After your OpenLane run finishes, you need to find your synthesized gate-level netlist (`.v`) and your Standard Delay Format (`.sdf`) file. Navigate to your `runs/RUN_DATE` folder:

For Ubuntu (OpenLane 1.x):

1. **Gate-Level Netlist:** Look inside `results/final/verilog/gl/` for your gate-level netlist (e.g., `alu.nl.v`).
2. **SDF File:** Look inside `results/final/sdf/multicorner/nom/` for your typical corner physical delay file (e.g., `alu.Typical.sdf`).

For Mac (LibreLane / OpenLane 2):

1. **Gate-Level Netlist:** Look inside the `final/nl/` directory for your gate-level netlist (typically named `alu.nl.v`).
2. **SDF File:** Look inside the `final/sdf/` directory for your delay file (e.g., `alu__nom_tt_025C_1v80.sdf`).

Create a new folder named `gls_sim` in your main project directory (`designs/alu/gls_sim`) and copy these two files into it. Also, copy your original `alu_tb.v` into this folder.

2.2 Step 1.2: Modify the Testbench for Gate-Level constraints

When synthesis tools convert your RTL into physical gates, they often reorder the module ports. To prevent port mismatch errors, we must use **Explicit Named Port Mapping**. Furthermore, we must inject the SDF delay file using the `$sdf_annotate` system task.

Open your `alu_tb.v` file and update it to match this structure:

```
`timescale 1ns/1ps
module alu_tb(

);
```

```
  reg clk;
  reg [3:0] a, b;
  reg [2:0] opcode;
  wire [3:0] result;


  alu uut(clk, a, b, opcode, result);

  always #5 clk = ~clk;

  initial begin
    $dumpfile("alu_gls.vcd");
    $dumpvars(1,alu_tb);

    clk = 0;
    // Inject the physical delays from the SDF file
    // (Ensure this filename exactly matches your copied .sdf file!
    //  Ubuntu users: "alu.sdf"
    //  Mac users: "alu__nom_tt_025C_1v80.sdf")
    $sdf_annotate("alu.sdf", uut);

    a = 4'b1100;
    b = 4'b0101;
    #10 opcode = 3'b000;
    #10 opcode = 3'b001;
    #20 $finish;
  end
endmodule


// Following is a complete set on a Mac system
`timescale 1ns/1ps
module alu_tb(
);
  reg clk;
  reg [3:0] a, b;
  reg [2:0] opcode;
  wire [3:0] result;
  integer i;

  always #5 clk = ~clk;

  alu uut(clk, a, b, opcode, result);
  initial begin
    $dumpfile("alu_4bit.vcd");
    $dumpvars(1,alu_tb);
    $monitor("Time = %t | A = %b | B = %b | Op = %b | Res = %b",$time,
       ↪ a, b, opcode, result);

    $sdf_annotate ("alu__nom_tt_025C_1v80.sdf" , uut );

    clk = 0;
    a = 4'b1100;
    b = 4'b0101;
    for(i=0; i<8; i=i+1) begin
      opcode = i;
      #10;
    end
```

```
    $finish;
  end
endmodule
```

2.3 Step 1.3: Find Your PDK Installation Path

Because the gate-level netlist uses physical SkyWater 130nm standard cells, the simulator needs their behavioral models. Depending on your installation (Docker, CIEL, or Volare), the PDK path varies.

To find the exact location of the library on your machine, run this search command in your terminal:

```
find ~/.ciel ~/.volare ~/OpenLane ~/Desktop -name "sky130_fd_sc_hd.v"
    ↪ 2>/dev/null
```

This will output the exact path. For example, on one machine it might look like this:

```
/home/khushwantsehra/.ciel/sky130A/libs.ref/sky130_fd_sc_hd/verilog/
    ↪ sky130_fd_sc_hd.v
```

Copy the directory path (everything before `/sky130A/...`) and set it as your `PDK_ROOT` environment variable. For example:

```
export PDK_ROOT=/home/khushwantsehra/.ciel
```

2.4 Step 1.4: Compile and Run Icarus Verilog

We will now compile the simulation. Note the following strict command requirements:

- `-gspecify`: Forces the compiler to process the physical silicon delay paths. Without this, the SDF annotation is ignored.
- `'-DUNIT_DELAY=#1'`: Must be wrapped in single quotes so the terminal does not mistake the `#` for a comment.
- The output flag (`-o gls_out`) must come **before** the Verilog file paths.
- Ensure you are targeting the correct netlist name (`alu.nl.v`).

Run this **exact command as one single continuous line** in your terminal:

```
iverilog -gspecify -o gls_out -Ttyp -DFUNCTIONAL '-DUNIT_DELAY=#1'
    ↪ $PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog/primitives.v
    ↪ $PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog/
    ↪ sky130_fd_sc_hd.v alu.nl.v alu_tb.v
```

Once compiled successfully, execute the simulation:

```
vvp gls_out
```

View the resulting `alu_gls.vcd` in GTKWave. If you zoom in closely on the clock edge, you will notice that the `result` signal does not change instantly. It shifts slightly a few picoseconds after the clock edge, proving your physical delays are correctly annotated!